Learning to Select Context for Code Review: Midpoint Progress Report

Michael Liu **Ruiding Feng** **Li-hua Peng**
Machine Learning II / CS536, Rutgers University

Abstract

Our project asks whether a learned selector can choose useful context for each code change while preserving a language model's review accuracy and reducing input tokens. The main study will keep the reviewer frozen and train a smaller context selector. At the midpoint, we have implemented repository-based data splitting, a preliminary Qwen3B adaptation workflow, checkpoint recovery, and a separate comparison notebook. This report describes that infrastructure, the planned baseline comparison, and lessons from cloud training. Quantitative evaluation remains pending; the result placeholders below do not establish an improvement in review quality or token efficiency.

Repository: `https://github.com/michaelsliu3/CS536_Project`
Milestone: November 2, 2026 midpoint check-in; draft with results pending.

1 Research question and current scope

Large language models can assist code review by examining proposed changes and related information. A change may need an issue description, a pull request (PR) description, or surrounding code to be understood. Supplying every field also increases the input length. ContextCRBench provides code changes with linked textual and code context [1]. Selective retrieval has also been studied for repository-level code completion [2]; whether a separate selector helps our code-review task remains an empirical question.

Our central question is: *Can a small trained selector choose context separately for each code change, maintaining review accuracy while using fewer tokens than fixed context choices?* Following instructor feedback, we focus on the accuracy–cost trade-off. Robustness is outside the core scope, and transfer to another reviewer is optional. The planned success criterion is accuracy within one percentage point of the best fixed strategy with at least 30% fewer extra context tokens, or higher accuracy at a matched token budget. These are targets, not measured outcomes.

The current Qwen3B fine-tuning run is a preliminary experiment for establishing the data, training, and evaluation workflow. It asks whether adapting a reviewer improves its merge-status predictions. This differs from the proposed main method, which keeps the reviewer unchanged and trains a context selector. Completing the pilot will not by itself answer the context-selection question.

2 Implemented pilot and evaluation protocol

Data and task. We use ContextCRBench and predict the Boolean merge-status label through a short JSON answer. Merge status is a proxy for code quality: a merged change is not necessarily defect-free, and a rejected change may have nontechnical reasons. Prompts include the code change and available issue, PR, and surrounding-code fields, while excluding the explicit label and review-answer fields. Field availability before the decision, indirect answer leakage, and near-duplicates still require manual audit.

Preprint.

The splitter targets 80/10/10 training/validation/test proportions *by repository*; row proportions can differ. It removes exact normalized-diff duplicates and excludes whole duplicate groups with conflicting labels. A saved manifest records membership and exclusions. Repository grouping reduces overlap between related changes across partitions, but does not establish temporal independence. The test partition is reserved for final evaluation after model and protocol choices are fixed. This is holdout validation, not cross-validation.

A simple Python example from the dataset. Training record `final_json/63859.json` comes from `pandas-dev/pandas`, PR #53773. The issue reports a `TypeError` when nested JSON contains `null`. The excerpt inside the recursive extraction function is copied from its diff; the header, function signature, outer context, and later lines are omitted. A leading + marks an added line.

```
+         if data is None:
+             return
          if isinstance(data, dict):
              data = [data]
```

The added check returns immediately for missing data; otherwise processing continues. Our model receives the change and available context and predicts `is_merged`. The recorded target here is `False`, kept out of the input; it does not prove that this check is incorrect. This is a data-format illustration, not a model prediction or a selected success case.

Model and training. The pilot uses Qwen2.5-Coder-3B-Instruct with quantized low-rank adaptation (QLoRA), which trains adapters through a frozen quantized base model [3]. The implemented configuration uses 4-bit NF4 with double quantization, adapter rank 16, scaling 32, and dropout 0.05. The learning rate is 2×10^{-4} with a cosine schedule and 3% warmup; weight decay is 0.01. Batch size one with eight accumulated batches gives an effective batch size of eight, and the maximum sequence length is 2,048 tokens. Loss is computed on the answer tokens. These describe the configured workflow; the submitted report must reconcile them with the saved cloud run configuration.

Training runs on a cloud NVIDIA GPU. We save model revision, package versions, configuration, and persistent metrics. Checkpoints retain adapter, optimizer, scheduler, random state, and progress; interrupted runs resume toward the saved epoch target. Extending a completed schedule is a separate experimental decision.

Comparison protocol. The evaluation notebook compares a constant classifier fixed from training-label counts, the untouched base model, and the same base plus the adapter. All use the same saved validation sample (up to 200 predetermined examples); the two language-model conditions share prompts, truncation, quantization, and deterministic decoding. Invalid JSON is counted as incorrect and reported separately. We will report accuracy, macro-F1, per-class scores, and malformed-output rate, then inspect paired improvements and regressions. Training-time validation loss and teacher-forced completion accuracy are diagnostics, not substitutes for independently generated answers.

3 Initial results: pending evaluation

No verified paired evaluation results are included in this draft. Table 1 reserves the required baseline comparison; no dash represents a zero score. Training progress or a decreasing loss alone would not establish better held-out predictions.

Table 1: Planned comparison on the same frozen validation sample. All performance entries are pending; the final test set is not used here.

Method	Accuracy	Macro-F1	Invalid JSON rate
Training-majority constant	—	—	N/A
Untouched Qwen3B	—	—	—
Qwen3B + trained adapter	—	—	—

[Pending: Insert evaluated sample size, class counts, checkpoint identity, completed epochs, and the three methods' scores.]

[Pending: Add a compact training/validation curve and discuss paired successes and failures, including negative results.]

4 Lessons learned, successes, and challenges

Successes and lessons. The main engineering progress is a reproducible path from dataset records to saved training state and a common evaluation interface. Freezing the split before training makes the three-way comparison interpretable. Separating evaluation from training allows us to inspect an intermediate adapter without retraining. Persistent logs and verified checkpoints also reduce dependence on a browser session; reconnecting to Jupyter does not necessarily restore its displayed output.

Challenges and changes. Conflicting labels for identical changes exposed ambiguity in the data, motivating exclusion of those groups instead of choosing a label arbitrarily. Cloud model-download timeouts motivated loading a local copy of the base weights. Checkpoint storage was moved off the nearly full system disk. A strict configuration check initially blocked harmless notebook edits; matching training configurations now resume automatically, while changed training settings select a separate run folder.

Frequent full validation also interrupted training throughput. We changed the interval from 100 to 1,000 optimizer steps while keeping frequent checkpoint saves. Resume initially restored the old validation interval, so the callback now reapplies the requested cadence after loading checkpoint state. These are workflow corrections, not measured accuracy or speed improvements. Longer prompts remain a concern: truncation can remove useful context, so nominally supplying all fields does not guarantee the model sees them all.

5 Next steps and remaining risks

The immediate priority is to complete the paired pilot evaluation, populate Table 1, and explain both gains and failures before the midpoint submission. We will verify the exact training configuration and data exclusions from saved artifacts, and add a compact loss plot and representative examples. A fixed validation sample will support development; the reserved test set will remain untouched during these decisions.

The next research phase will compare diff-only, all-context, and fixed context combinations using a frozen reviewer, then add random and similarity-based selection. On training repositories only, cached reviewer outcomes will supply targets for a compact learned selector. We will measure accuracy and macro-F1 against extra context tokens and include selector overhead in runtime. A small pilot should establish whether context choices change predictions enough to justify larger target generation. If the selector fails to improve on the best fixed strategy, we will analyze where context helps or hurts. Repeated seeds and context-type ablations are planned, not completed.

Acknowledgments and tools. We thank Professor Lesia Semenova for guidance on scope and evaluation. OpenAI Codex assisted with implementation, debugging, and this draft. The report uses the supplied NeurIPS style and repository writing skills; the team remains responsible for verifying code, sources, results, and final claims.

References

[1] R. Hu et al. Benchmarking LLMs for fine-grained code review with enriched context in practice. *arXiv:2511.07017*, 2025. `https://arxiv.org/abs/2511.07017`.

[2] D. Wu, W. U. Ahmad, D. Zhang, M. K. Ramanathan, and X. Ma. Repoformer: Selective retrieval for repository-level code completion. *ICML*, PMLR 235:53270–53290, 2024.

[3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. *arXiv:2305.14314*, 2023.